INPUT & OUTPUT DEVICES

Structured Exam Notes

Computer Graphics (Input & Interaction) | Virtual Reality (Input & Output Devices)

Syllabus Coverage Map

Syllabus: Input and Interaction: Input Devices. Physical Input Devices, Logical Devices. Measure and trigger. Input Modes. Event-Driven Input: Using the pointing device, Window events, and Keyboard events. Menus. VR related Input Devices: Trackers, Navigation, and Gesture Interfaces; VR related Output Devices: sound displays & haptic feedback.

Syllabus topic	Section	Source
Input Devices, Physical Input Devices	A.1 – A.3	CG Unit 2 slides
Logical Devices	A.4	CG Unit 2 slides
Measure and Trigger	A.5	CG Unit 2 slides
Input Modes	A.6	CG Unit 2 slides
Event-Driven Input (pointing device, window events, keyboard events)	A.7 – A.10	CG Unit 2 slides
Menus	A.11	CG Unit 2 slides
VR Input: Trackers	B.1 – B.3	VR Interfaces PPT + Ch. 2 (Burdea)
VR Input: Navigation (& manipulation) interfaces	B.4	VR Interfaces PPT + Ch. 2
VR Input: Gesture interfaces	B.5	VR Interfaces PPT + Ch. 2
VR Output: Sound displays	C.1 – C.2	VR Interfaces PPT + Ch. 3
VR Output: Haptic feedback	C.3	VR Interfaces PPT + Ch. 3

> **NOTE:** Topics in the uploaded files that are **not in your syllabus** (HMD / graphics displays, picking, display lists, double buffering, rendering modes, OpenGL primitives) are deliberately left out. Textbook extras that explain syllabus topics (3D probes, sensor coupling, haptic receptors) are included.

PART A: INPUT AND INTERACTION (Computer Graphics)

A.1 Introduction to Interactive Graphics

Objectives: introduce the basic input devices — Physical Devices, Logical Devices, Input Modes — then Event-driven input and Programming event input with GLUT.

Project Sketchpad

Ivan Sutherland (MIT, 1963) established the basic interactive paradigm that characterizes interactive computer graphics:

1. User sees an object on the display
2. User points to (**picks**) the object with an input device (light pen, mouse, trackball)
3. Object changes (moves, rotates, morphs)
4. Repeat

Graphical Input — Three Ways to Describe It

Aspect	Description	Examples
Physical properties	The actual hardware device	Mouse, keyboard, trackball
Logical properties	What is returned to the program via the API	A position, an object identifier
Modes	How and when input is obtained	Request or event

A.2 Physical Input Devices

Common physical devices: **mouse, trackball, light pen, data tablet, joystick, space ball**.

Incremental (Relative) Devices

- Devices such as the **data tablet** return a **position directly** to the operating system.
- Devices such as the **mouse, trackball, and joystick** return **incremental inputs (or velocities)** to the operating system.
- Must **integrate** these inputs to obtain an absolute position:
  - Rotation of cylinders in mouse
  - Roll of trackball
  - Difficult to obtain absolute position
  - Can get variable sensitivity

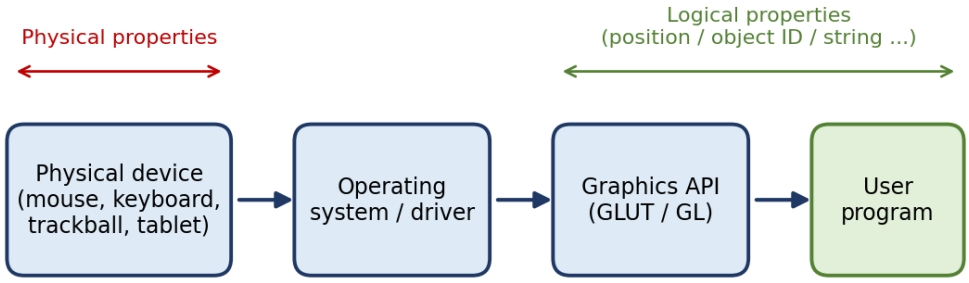
	Absolute device	Incremental (relative) device
Returns	Position directly	Increments / velocities
Example	Data tablet	Mouse, trackball, joystick
Absolute position	Available directly	Must be obtained by integrating inputs
Sensitivity	Fixed	Variable sensitivity possible

A.3 Logical Devices

Consider the C and C++ code:

```
C++:  cin >> x;
C:    scanf("%d", &x);
```

- What is the input device? **Can't tell from the code** — could be keyboard, file, output from another program.
- The code provides **logical input**: a number (an int) is returned to the program **regardless of the physical device**.



Program sees only WHAT is returned - not WHICH device produced it

Fig A.1 Physical device vs logical device

Graphical Logical Devices

Graphical input is more varied than input to standard programs, which is usually numbers, characters, or bits. Two older APIs (**GKS, PHIGS**) defined **six types of logical input**:

Logical device	Returns	Typical physical device [added example]
Locator	A position	Mouse, tablet
Pick	ID of an object	Light pen, mouse click on object
Keyboard	Strings of characters	Keyboard
Stroke	Array of positions	Mouse drag / tablet stroke
Valuator	Floating point number	Slider, dial
Choice	One of n items	Menu, function buttons

X Window Input

The **X Window System** introduced a **client-server model** for a network of workstations:

- **Client:** OpenGL program
- **Graphics Server:** bitmap display with a pointing device and a keyboard

> **KEY POINT:** Logical devices hide the hardware: the program asks for a *kind* of input (position, ID, string…) and the system supplies it from whatever physical device is present.

A.4 Measure and Trigger

- Input devices contain a **trigger** which can be used to send a signal to the operating system:
  - Button on mouse
  - Pressing or releasing a key
- When triggered, input devices return information (their **measure**) to the system:
  - Mouse returns **position** information
  - Keyboard returns **ASCII code**

Device	Trigger	Measure
Mouse	Button click	Position information
Keyboard	Press / release of a key	ASCII code

> KEY POINT: **Trigger** = the action that sends the signal. **Measure** = the data returned when triggered.

A.5 Input Modes

Input modes describe **how and when** input is obtained: **Request mode** or **Event mode**.

Request Mode

- Input is provided to the program **only when the user triggers the device**.
- Typical of **keyboard input**: can erase (backspace), edit, correct **until enter (return) key (the trigger) is pressed**.

Event Mode

- Most systems have **more than one input device**, each of which can be triggered at an **arbitrary time** by a user.
- Each trigger generates an **event** whose measure is put in an **event queue** which can be examined by the user program.

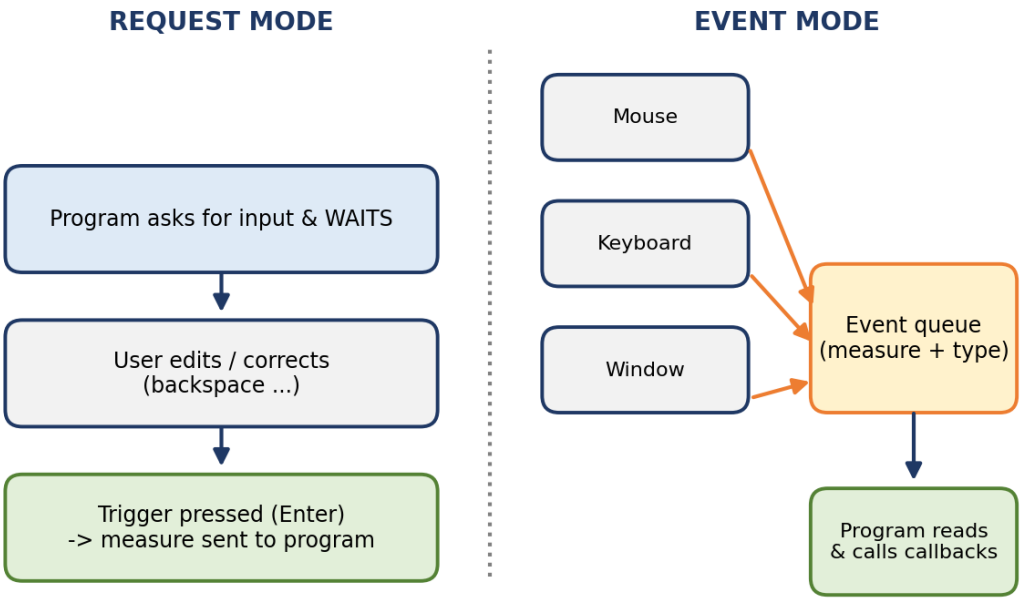


Fig A.2 Request mode vs Event mode

	Request mode	Event mode
Who controls timing	Program (waits for input)	User (arbitrary time)
Input delivered	Only when trigger is pressed	Each trigger creates an event placed in a queue
Typical device	Keyboard (editing until Enter)	Mouse, keyboard, window system
Number of devices	Usually one at a time	Several devices active together

Event Types

Event	Meaning
Window	resize, expose, iconify
Mouse	click one or more buttons
Motion	move mouse
Keyboard	press or release a key
Idle	no event — define what should be done if no other event is in queue

A.6 Event-Driven Input and Callbacks

Callbacks

- **Programming interface for event-driven input.**
- Define a **callback function for each type of event** the graphics system recognizes.
- This user-supplied function is **executed when the event occurs**.

GLUT example:

```
glutMouseFunc(mymouse);   // mymouse = mouse callback function
```

GLUT Callbacks

GLUT recognizes a **subset** of the events recognized by any particular window system (Windows, X, Macintosh).

Callback registration	Event handled
`glutDisplayFunc`	Window needs refreshing (display callback)
`glutMouseFunc`	Mouse button press / release
`glutMotionFunc`	Mouse moves with a button pressed
`glutPassiveMotionFunc`	Mouse moves with no button pressed
`glutKeyboardFunc`	Key pressed
`glutReshapeFunc`	Window resized / reshaped
`glutIdleFunc`	No events in queue

GLUT Event Loop

The last line in `main.c` for a program using GLUT must be `glutMainLoop();` which puts the program in an **infinite event loop**. In each pass through the event loop, GLUT:

1. looks at the events in the queue
2. for each event in the queue, executes the appropriate callback function **if one is defined**
3. if no callback is defined for the event, the event is **ignored**

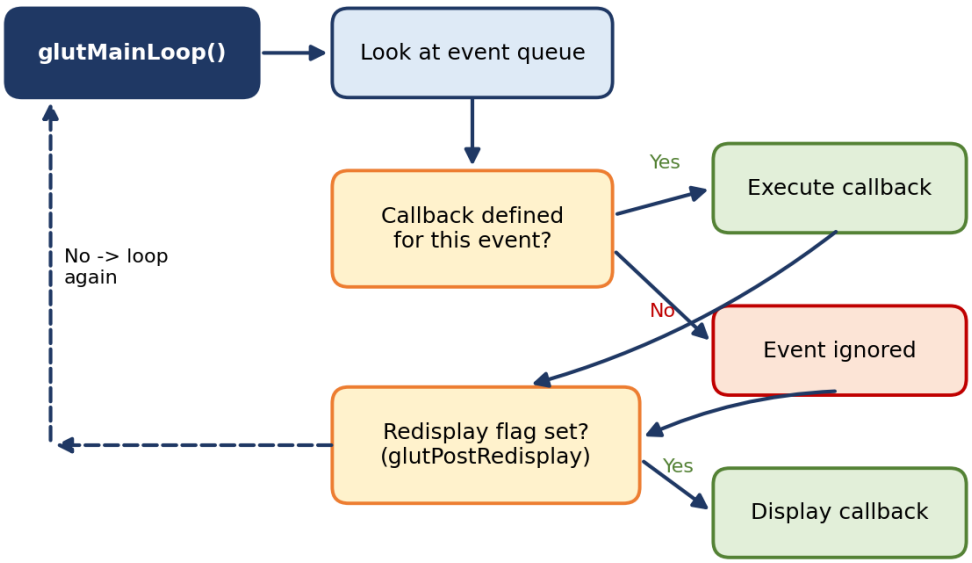


Fig A.3 GLUT event loop

The Display Callback

Executed whenever GLUT determines that the window should be refreshed, for example:

- When the window is first opened
- When the window is reshaped
- When a window is exposed

- When the user program decides it wants to change the display

In `main.c`: `glutDisplayFunc(mydisplay)` identifies the function to be executed. **Every GLUT program must have a display callback.**

Posting Redisplays

- Many events may invoke the display callback function → can lead to **multiple executions** of the display callback on a single pass through the event loop.
- Avoid this by using `glutPostRedisplay();` which **sets a flag**.
- GLUT checks to see if the flag is set at the **end of the event loop**; if set, the display callback is executed.

Idle Callback and Globals

- The **idle callback** is executed whenever there are **no events in the event queue**: `glutIdleFunc(myidle)`. Useful for animations.

```
void myidle() {
    /* change something */
    t += dt;
    glutPostRedisplay();
}
void mydisplay() {
    glClear(GL_COLOR_BUFFER_BIT);
    /* draw something that depends on t */
    glutSwapBuffers();
}
```

- The **form of all GLUT callbacks is fixed** (e.g. `void mydisplay()`, `void mymouse(GLint button, GLint state, GLint x, GLint y)`), so **globals** must be used to pass information to callbacks (e.g. `float t; /* global */`).

> **KEY POINT:** Event-driven programming = register callbacks → call `glutMainLoop()` → GLUT dispatches each queued event to its callback.

A.7 Using the Pointing Device (Mouse Events)

The Mouse Callback

```
glutMouseFunc(mymouse);
void mymouse(GLint button, GLint state, GLint x, GLint y)
```

Returns:

- **which button** (`GLUT_LEFT_BUTTON`, `GLUT_MIDDLE_BUTTON`, `GLUT_RIGHT_BUTTON`) caused the event
- **state** of that button (`GLUT_UP`, `GLUT_DOWN`)
- **position** in window (x, y)

Positioning — Coordinate Inversion

- The position in the screen window is usually measured in **pixels with the origin at the top-left corner** — a consequence of refresh being done from top to bottom.
- OpenGL uses a world coordinate system with the **origin at the bottom left**.

- Therefore the y coordinate returned by the callback must be inverted using the height of the window: **`y = h – y;`**

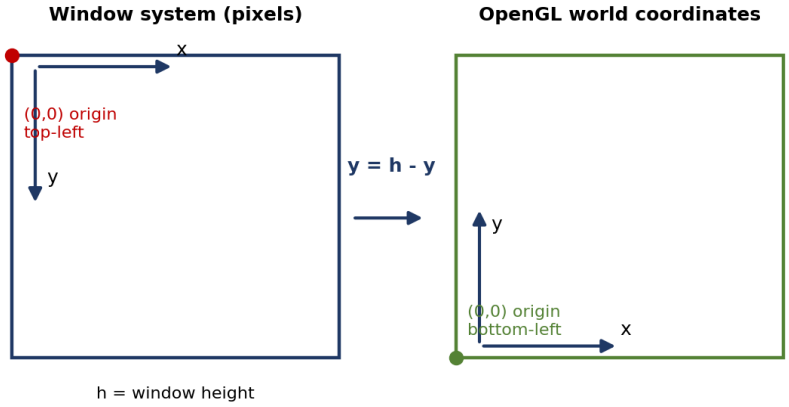


Fig A.4 Window (pixel) coordinates vs OpenGL coordinates

Terminating a Program

```
void mouse(int btn, int state, int x, int y)
{
    if (btn == GLUT_RIGHT_BUTTON && state == GLUT_DOWN)
        exit(0);
}
```

Using the Mouse Position — Drawing Squares at Cursor Location

A small square is drawn at the location of the mouse each time the left mouse button is clicked. This example does not use the display callback, but one is required by GLUT, so an **empty display callback** `mydisplay(){}` is used.

```
void mymouse(int btn, int state, int x, int y)
{
    if (btn == GLUT_RIGHT_BUTTON && state == GLUT_DOWN)
        exit(0);
    if (btn == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
        drawSquare(x, y);
}

void drawSquare(int x, int y)
{
    y = h - y;   /* invert y position  [slide writes w - y; h = window height] */
    glColor3ub((char)rand()%256, (char)rand()%256, (char)rand()%256); /* random colour */
    glBegin(GL_POLYGON);
        glVertex2f(x+size, y+size);
        glVertex2f(x-size, y+size);
        glVertex2f(x-size, y-size);
        glVertex2f(x+size, y-size);
    glEnd();
}
```

Motion Callbacks

Callback	Effect
`glutMotionFunc(drawSquare)`	Draw squares (or anything else) **continuously as long as a mouse button is pressed**
`glutPassiveMotionFunc(drawSquare)`	Draw squares **without pressing a button**

A.8 Keyboard Events

```
glutKeyboardFunc(mykey);
void mykey(unsigned char key, int x, int y)
{
    if (key == 'Q' || key == 'q')
        exit(0);
}   /* slide shows void mykey() and '|' – corrected */
```

- Returns the **ASCII code of the key pressed** and the **mouse location**.

Special and Modifier Keys

- GLUT defines the **special keys** in `glut.h`: Function key 1 → `GLUT_KEY_F1`; Up arrow key → `GLUT_KEY_UP`.
- Can also check whether one of the **modifiers** is pressed using `glutGetModifiers()`:

Modifier constant	Key
`GLUT_ACTIVE_SHIFT`	Shift
`GLUT_ACTIVE_CTRL`	Ctrl
`GLUT_ACTIVE_ALT`	Alt

> **KEY POINT:** Modifiers allow **emulation of a three-button mouse with a one- or two-button mouse**.

A.9 Window Events (Reshaping the Window)

- The OpenGL display window can be reshaped/resized by pulling the corner of the window. What happens to the display? It **must be redrawn by the application**.
- Two possibilities:
  - Display **part of the world**
  - Display the **whole world but force it to fit** in the new window → can alter the aspect ratio

The Reshape Callback

```
glutReshapeFunc(myreshape);
void myreshape(int w, int h)
```

- Returns the **width and height of the new window (in pixels)**.
- A **redisplay is posted automatically** at the end of execution of the callback.
- GLUT has a default reshape callback but you probably want to define your own.
- The reshape callback is a **good place to put viewing functions** because it is invoked when the window is first opened.

Example Reshape — preserves shapes (viewport and world window have the same aspect ratio)

```
void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);   /* switch matrix mode */
    glLoadIdentity();
    if (w <= h)
        gluOrtho2D(-2.0, 2.0, -2.0*(GLfloat)h/(GLfloat)w, 2.0*(GLfloat)h/(GLfloat)w);
    else
        gluOrtho2D(-2.0*(GLfloat)w/(GLfloat)h, 2.0*(GLfloat)w/(GLfloat)h, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);    /* return to modelview mode */
}
```

A.10 Menus

Toolkits and Widgets

- Most window systems provide a **toolkit** or library of functions for building user interfaces that use special types of windows called **widgets**.
- Widget sets include tools such as **Menus, Slidebars, Dials, Input boxes**.
- But toolkits tend to be **platform dependent**; **GLUT provides a few widgets including menus**.

GLUT Pop-up Menus

GLUT supports **pop-up menus**; a menu can have **submenus**. Three steps:

1. Define entries for the menu
2. Define action for each menu item (action carried out if entry selected)
3. Attach menu to a mouse button

Defining a Simple Menu (in main.c)

```
menu_id = glutCreateMenu(mymenu);
glutAddMenuEntry("clear Screen", 1);   /* entry text, identifier */
glutAddMenuEntry("exit", 2);
glutAttachMenu(GLUT_RIGHT_BUTTON);        /* entries appear when right button pressed */
```

> **NOTE:** The slide writes `glutAddmenuEntry` / `gluAddMenuEntry` — the correct name is **`glutAddMenuEntry`**.

Menu Actions (Menu Callback)

```
void mymenu(int id)
{
    if (id == 1) glClear(GL_COLOR_BUFFER_BIT);   /* slide: glClear() */
    if (id == 2) exit(0);
}
```

- Each menu has an **id that is returned when it is created**.
- Add submenus by: `glutAddSubMenu(char *submenu_name, submenu_id)` — creates an **entry in the parent menu**.

Function	Purpose
`glutCreateMenu(callback)`	Create menu, returns menu id; callback receives the selected entry id
`glutAddMenuEntry(name, id)`	Add an entry (text shown + identifier)
`glutAddSubMenu(name, submenu_id)`	Add a submenu as an entry in the parent menu
`glutAttachMenu(button)`	Attach menu to a mouse button (e.g. `GLUT_RIGHT_BUTTON`)

Other Functions in GLUT

- Dynamic windows (create and destroy during execution)
- Subwindows
- Multiple windows
- Changing callbacks during execution
- Timers
- Portable fonts: `glutBitmapCharacter`, `glutStrokeCharacter`

> **KEY POINT:** Mini summary (Part A): Devices are *physical* or *logical*; they return a **measure** when their **trigger** fires; input is obtained in **request** or **event** mode; GLUT handles events through **callbacks** (display, mouse, motion, keyboard, reshape, idle, menu) dispatched by `glutMainLoop()`.

PART B: VR RELATED INPUT DEVICES

B.1 Introduction

One of the three I's defining virtual reality stands for **interactivity**. In order to allow human–computer interaction it is necessary to use special interfaces designed to **input a user's commands into the computer** and to **provide feedback from the simulation to the user**. VR interfaces are varied in functionality and purpose, as they address several human sensorial channels. For example:

- body motion is measured with **3D position trackers** or using sensing suits
- hand gestures are digitized by **sensing gloves**
- visual feedback is sent to **stereo HMDs** and large-volume displays
- virtual sound is computed by **3D sound generators**

The aim of researchers is to allow faster and more natural ways of interaction with the computer and thus overcome the communication bottleneck presented by the keyboard and the computer mouse.

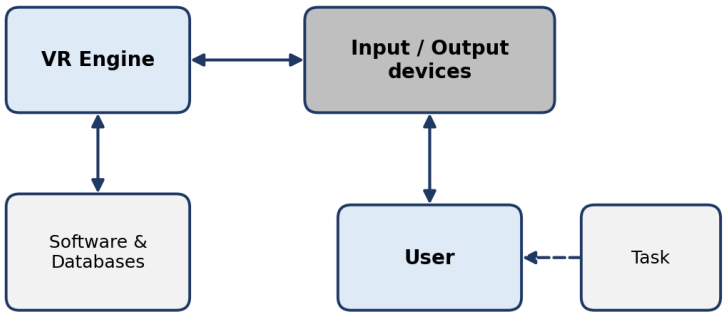


Fig B.1 VR system architecture (VR engine, I/O devices, user and task)

Input devices covered: Trackers → VR Navigation → Gesture Interface.

B.2 3D Position Trackers

- The **need** is **real-time position and orientation of a moving object within some reference frame**.
- **Applications:** Robotics, missile tracking (ballistic missile tracking), navigation, ubiquitous computing, biomechanics, CAD, architecture, education, and VR.
- **Varying requirements** of parameters like **measurement range, precision and temporal update rates**.
- **Example:** GPS transducers report position with an accuracy of **1–5 m (95% of the time)** on a worldwide scale — good for **car navigation**.
- **VR focuses on more stringent accuracy requirements**; on the other hand the scale of measured distances for VR is usually **not larger than a room or a laboratory**.

Six Degrees of Freedom

- A moving object in 3D has **6 degrees of freedom**: **3 translations and 3 rotations**.

- If a Cartesian coordinate system is attached to the moving object, translations are along the **X, Y and Z** axes.
- Rotations about the **x, y and z axes** are called **yaw, pitch and roll** respectively.
- These define a dataset of **six numbers** that need to be measured sufficiently rapidly, as the object may be moving at high speed.

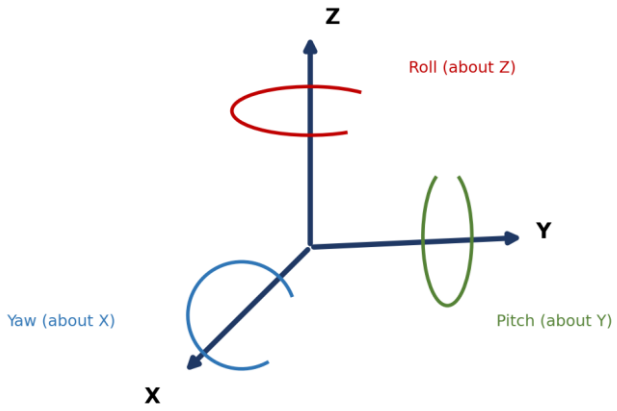


Fig B.2 Coordinate system of a moving 3D object (6 DOF: X, Y, Z translation; yaw, pitch, roll rotation)

> **DEFINITION: Tracker:** the special-purpose hardware used in VR to measure the real-time change in a 3D object position and orientation.

VR applications typically measure the motion of the user's **head, hands, or limbs** for the purpose of **view control, locomotion, and object manipulation**. A newer application is the control of an **avatar** (virtual body) mapped to the user.

Tracking Technologies

Several competing tracking technologies are available: **mechanical, magnetic, optical, ultrasonic, and hybrid**. It is therefore necessary to look at tracker performance parameters in order to match their measurement capabilities to different sensorial channel requirements and available budgets.

HMD Tracker (Example Application)

- **Tracker receiver is placed on the HMD** (on the user's head), so that when head posture changes, so does the position of the receiver.
- User's head motion is **sampled and sent as electronic signal to the host computer** (graphics workstation).
- The computer uses the tracker data to calculate a **new viewing direction** of the virtual scene and to render an updated image (converted to NTSC video signals displayed on the two LCD screens).
- 3D tracker required to change the spatial view according to head orientation: **immersion sensation achieved**. Without it, the spatial view could not match the user's head posture and the "immersion" sensation would be lost.
- Another sensorial modality is **3D sound presented in headphones**.
- **Co-locate sound sources with virtual objects** the user sees in the simulation — increases realism and the feeling of immersion.

- **Accuracy requirement for 3D sound is less than the 3D graphics** (visual acuity is higher than auditory localization acuity; auditory depth perception is even weaker).

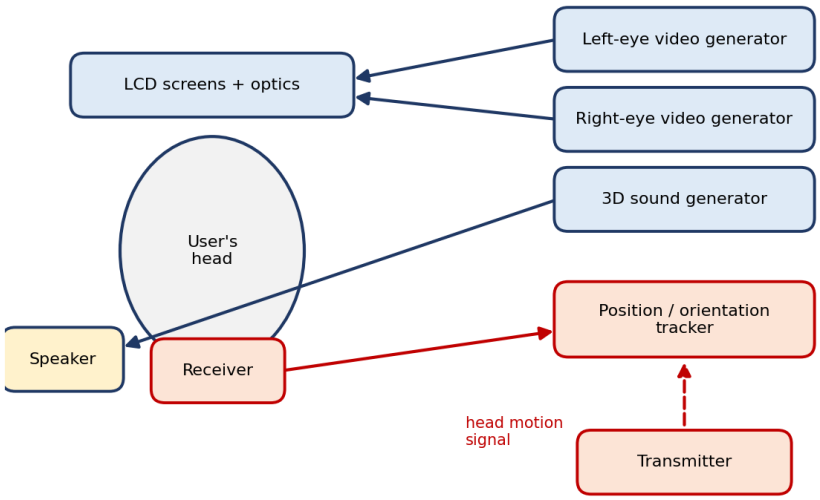


Fig B.3 3D magnetic tracker in a HMD (transmitter, receiver, tracker unit, video and 3D sound generators)

B.3 Tracker Performance Parameters

All 3D trackers, regardless of the technology they use, share a number of very important performance parameters: **Accuracy (translation & rotation), Jitter, Drift and Latency** — measured **with respect to distance and time**. **The smaller the difference, the better the performance.**

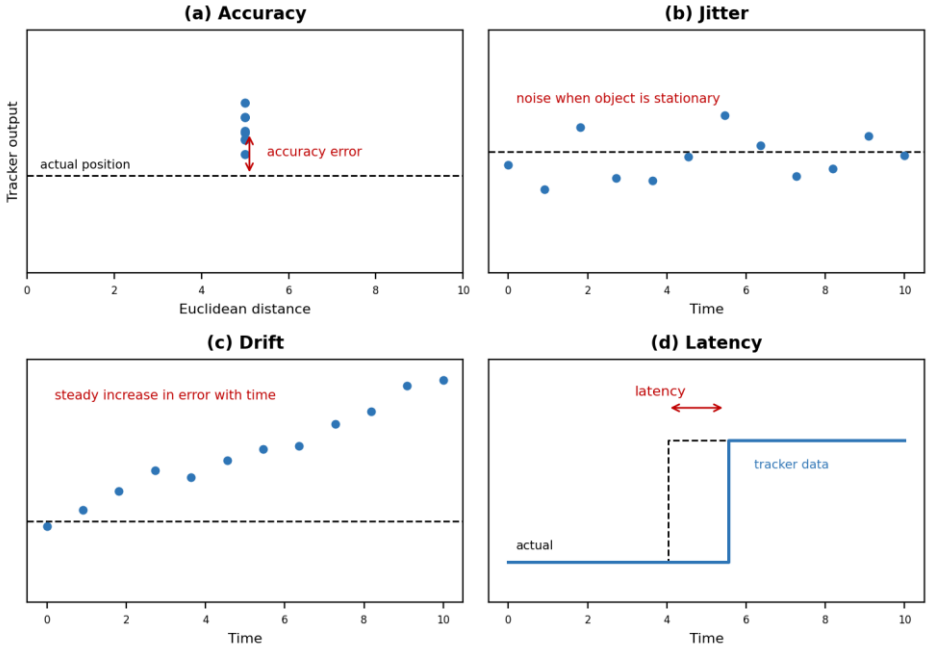


Fig B.4 Tracker performance parameters: (a) accuracy; (b) jitter; (c) drift; (d) latency

Parameter	Definition	Notes
Accuracy	Difference between the object's **actual 3D position** and that **reported by tracker measurements**.	The more accurate, the smaller this difference and the better the simulation follows the real user's actions. Given **separately for translation (fraction of a millimetre) and rotation (fraction of a degree)**. Typically **not constant** — degraded with distance from the origin of the reference frame. [Figure also shows *resolution* and *repeatability*.]
Jitter	The change in tracker output when the tracked object is **stationary** (noise).	A tracker with no jitter would report a constant value for a stationary object. Not constant over the work envelope; influenced by environmental conditions.
Drift	The **steady increase in tracker error with time**.	Output of a stationary object keeps growing away from the actual value. Drift is controlled by correcting with a secondary sensor — typical of **hybrid trackers**.
Latency	The **time delay between action and result**; for 3D trackers, the time between the **change in object position/orientation** and the time the sensor **detects this change**.	**Minimal latency is desired** — large latency causes "simulation sickness" (headaches, nausea) as the HMD view lags the user's head motion. System latency also includes communication delay and the time for the computer to render and display the scene.

KEY POINT: Accuracy → how *correct* the reading is. Jitter → how *noisy* it is at rest. Drift → how it *wanders* over time. Latency → how *late* it is. (**mnemonic: A-J-D-L**)

B.4 VR Navigation (Navigation and Manipulation Interfaces)

DEFINITION: A **navigation/manipulation interface** is a device that allows the **interactive change of the view** to the virtual environment and **exploration through the selection and manipulation of a virtual object of interest**.

Absolute vs Relative Coordinates

Navigation/manipulation can be done either through **absolute coordinates** or **relative coordinates**.

	Absolute (trackers)	Relative (trackballs etc.)
Reference	Position & orientation w.r.t. a **fixed system of coordinates** (transmitter-fixed)	Incremental control relative to the object's **previous 3D position**
Output at rest	Never a zero set (even if receiver is at rest)	**Always returns zeros if not acted upon**
Control of VR object	Position directly mapped to the absolute position of the receiver	Position of the VR object is **incremented by six signed quantities** at every simulation cycle

	Absolute (trackers)	Relative (trackballs etc.)
Velocity	—	Indirectly controlled: a **larger translation/rotation increment → larger velocity** of the VR object

Drawback of Absolute Position Control & Solutions

- Drawback: the user's **limited arm reach**; the larger the display, the bigger the problem.
- **Solution 1 – Gain:** multiply tracker readings by a constant (gain) larger than unity. Drawback: **tracker noise is equally amplified**, so selected objects appear more jittery.
- **Solution 2 – Indexed motion:** virtual objects are selected and deselected repeatedly. When deselected the object stays in place while the user moves the arm back, then reselects and moves it again.

B.4.1 Tracker-Based Navigation / Manipulation Interfaces

- Trackers with **user-programmable push buttons** become navigation/manipulation interfaces.
- Examples: **Polhemus 3Ball**, **Ascension Technology 3D Mouse**.

Device	Description / use
3Ball	Hollow billiard ball housing a tracker receiver, with a pushbutton on the surface. A **virtual camera moves along the 3D vector** controlled by the ball as long as the pushbutton is pressed (object selected when pushbutton pressed).
Wand	**Projects a virtual ray along the tracker-controlled 3D vector**. Any object intersected by the ray is **selected by pushing the button**, then repositioned through wrist motion.
3D Mouse	Works both as a **mouse (2D)** and as a navigation/manipulation interface in **3D** (like the 3Ball); further functionality via programming of its **three buttons**.

Cubic Mouse

- 3 translating rods, **6 pushbuttons**; designed for **dual-hand operation** (Fröhlich et al., 2000).
- **Hold the mouse with the non-dominant hand** and push the rods and buttons using the dominant hand.
- Plastic cube **9 cm × 9 cm × 9 cm**; houses a **Polhemus Fastrak** (magnetic tracker), three **mutually perpendicular translating rods**, and six application-programmable buttons.
- Opposed to wands (designed for **flyby navigation**), the Cubic Mouse is designed primarily for **manipulation of a single, large virtual model**: rotating/translating the model through the tracker and **dissecting/editing it using the three rods**.
- Each rod controls a **cutting plane**; buttons at the end of the rods select the **orientation of the cutting planes**.
- Control buttons: one **clutch button** (indexed motion) and **two zoom buttons** (zoom in–out).
- **Proprioception** (the body's own sense of position/motion) and tactile cues allow operation **without looking at the rods**.

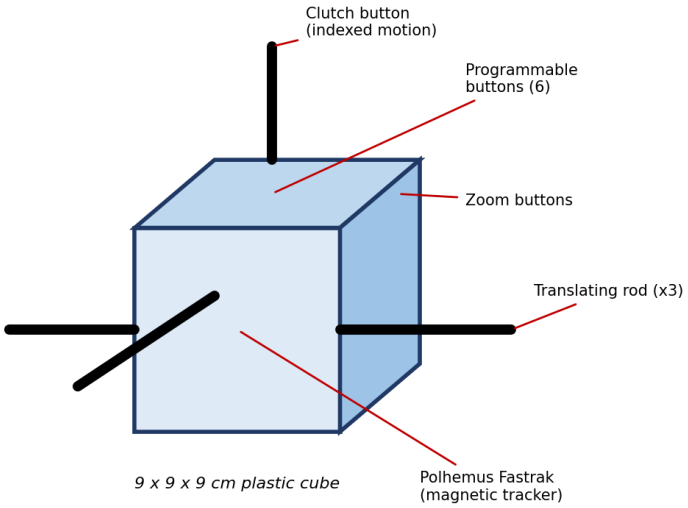


Fig B.5 The Cubic Mouse

B.4.2 Trackballs — N-M Interface

- Work on **relative coordinates** (e.g. Logitech **Magellan**).
- **Sensorized cylinder** that measures **three forces and three torques** applied by the user's hand on a compliant element.
- Force and torque are measured indirectly using the **Spring Deformation Law**.
- **Center is fixed with 6 LEDs**; **moving external cylinder with 6 photo sensors**.
- Force and torques are applied on the moving shell; **photo sensor output is used to measure** them.
- Measured values are sent to the host using an **RS232 serial line**; there they are multiplied by **software gains** to return a differential change in the controlled object's position and orientation.

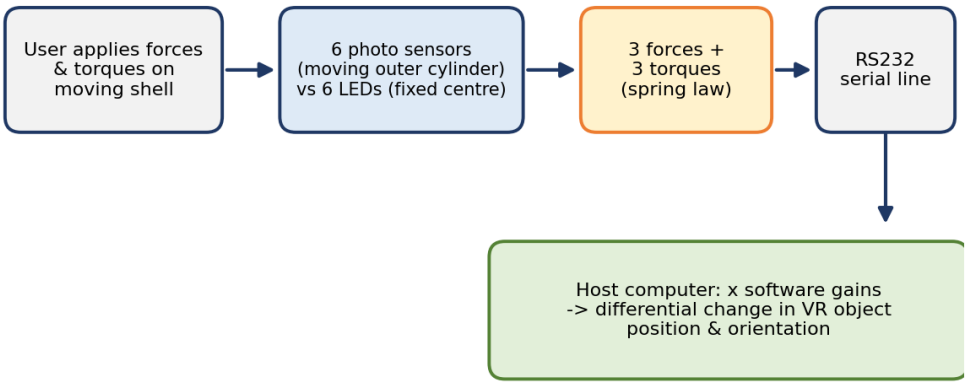


Fig B.6 Trackball (Magellan) sensing and data flow

- **Larger gains → larger speeds** for the VR object, but motion will not be smooth if the host cannot refresh the screen fast enough.
- **Force control** (alternative): forces measured by the trackball control forces applied by the VR object on the simulated environment. Can also be used to **fly by** — the sensor affects velocity and orientation of a virtual camera.

- **Pushbuttons** (binary on/off, programmable) within reach of the fingers: increase simulation gains (VR object speed); couple/decouple trackball with the VR object; start/stop simulation; reset to a default start location if the user becomes disoriented.
- **Drawback — sensor coupling:** the user may want pure translation but the object may translate and rotate (non-zero sensed torques when pure forces are applied by the fingers). Suppressed by **software filters** that read only forces, or **hardware filters** (pushbuttons to select translation-only, rotation-only, or dominant-motion input).

B.4.3 Three-Dimensional Probes

- An I/O device that is **intuitive, inexpensive**, and allows **absolute or relative** position control: the **Immersion Probe** (early 1990s) → renamed **MicroScribe 3D**; later also used to **digitize objects**.
- A small **sensorized mechanical arm** on a support base (6 in. × 6 in. footprint) with **six joints (joints 0–5)**; each rotary joint = **one DOF → 6 DOF** (simultaneous positioning and orienting of the tip).
- A **counterbalance** near the base minimizes user fatigue.
- Tip position relative to the base is obtained by **direct kinematics** from joint sensor values and link lengths; software reads joint sensors over **RS232**.
- A **binary switch on a foot pedal** selects/deselects virtual objects, starts/stops navigation, or marks a point on a real object surface for digitization; it may also control **flying speed** (velocity direction = probe-tip orientation).
- Workspace: **50-in-radius hemisphere** (models 3D, 3DX) to **66-in-radius hemisphere** (3DL, 3DLX). **Not affected by magnetic fields** (mechanical); over **1000 positions/sec with minimal latency**.
- **Accuracy errors:** joint sensor resolution, temperature variations, joint tolerances; **open kinematic chains accumulate errors from base toward tip** → the larger the model, the less accurate. Needs **calibration** (probe tip placed in a housing near the base). Model 3DX: **0.23 mm**; 3DLX: **0.30 mm**.

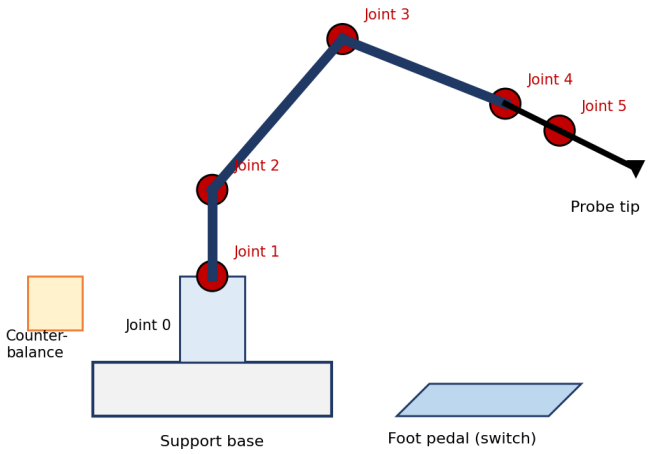


Fig B.7 MicroScribe 3D probe (schematic)

KEY POINT: Mini summary: Trackers give **absolute** 6-DOF data; add programmable buttons → 3Ball, wand, 3D mouse, Cubic Mouse. **Trackballs** give **relative** force/torque data scaled by gains. **Probes** are mechanical arms with joint sensors.

B.5 Gesture Interfaces

Why Gesture Interfaces?

- **Limitation of other interfaces** (trackballs, 3D probes): they limit the freedom of the user's hand to a **small area close to the desk**; natural motion of the hand is sacrificed and interaction is less intuitive. They do have the advantage of simplicity, compactness, and quiet operation.
- For **large gesture-based interactions**, I/O devices need to provide **free movement of the hand within a certain volume**.
- It is also desirable to allow **sensing of individual finger motions**.

> **DEFINITION: Gesture interfaces** are devices that measure the **real-time position of the user's fingers (and sometimes wrist)** in order to allow natural, gesture-recognition-based interaction with the virtual environment.

Sensing Gloves

- Most gesture interfaces today are **sensing gloves** that have **embedded sensors** to measure **finger movements** (position of each finger versus the palm).
- Gloves differ in: **type of sensors, number of sensors** (one or several per finger), **sensor resolution, glove sampling rate**, and whether they are **tethered or wireless**.
- Examples: **Fakespace Pinch Glove, 5DT Data Glove, Didjiglove, Immersion CyberGlove**. They measure some (or all) finger joint angles; some have **built-in trackers** to measure wrist motion.

Hand and Finger Motion Terminology

Motion	Description
Flexion – Extension	Finger bending / straightening
Abduction – Adduction	Lateral movement of fingers (palmar abduction, radial abduction)
Anteposition (opposition) – Retroposition	Thumb motion that brings it **in opposition to the palm**

Human fingers have degrees of freedom associated with flexion–extension and lateral abduction–adduction; additionally, the thumb has an anteposition–retroposition motion.

Work Envelope

The resulting sensing glove work envelope is **much larger than that of joysticks or trackballs**.

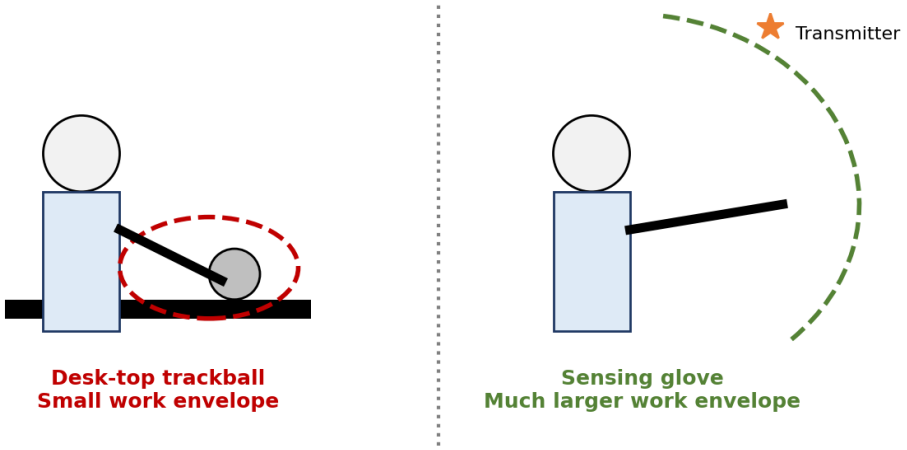


Fig B.8 Sensing glove work envelope vs desktop trackball work envelope

	Trackballs / 3D probes	Sensing gloves
Interaction	Single-point	**Dextrous, multipoint** (fingertips or palm)
Work envelope	Small area close to desk	Much larger volume
Realism	Less intuitive	More realistic, especially for **object manipulation** tasks
Navigation	Direct	Can become navigation interfaces via **user-programmed gesture libraries**

PART C: VR RELATED OUTPUT DEVICES

C.1 Introduction to Output Devices

Output (feedback) interfaces provide feedback from the simulation in response to user input. The sensorial channels fed back are **sight** (graphics displays), **sound** (3D sound displays), and **touch** (haptic displays). Combining several types of sensorial feedback **increases the user's immersion** and **improves simulation realism**, and therefore the usefulness of VR applications.

Output devices covered in this syllabus: 3D Sound and Haptic Display.

C.2 Sound Displays

> **DEFINITION: Sound displays** are computer interfaces that provide **synthetic sound feedback** to users interacting with the virtual world. The sound can be **monaural** (both ears hear the same sound) or **binaural** (each ear hears a different sound).

- Sound displays play an important role in increasing simulation realism by **complementing the visual feedback**.
- **By adding sound, the user's interactivity, immersion and perceived image quality increase.**

Example — Bouncing Ball

Situation	Sound needed	Why
User looks at a virtual ball bouncing in a virtual room on a **CRT monitor**	**Monaural** is sufficient	The ball is always in front of the user, displayed by the monitor
User wears an **HMD** and the ball bounces **outside the field of view**	**Binaural** is required	User cannot tell where the ball went from visual or monaural sound alone; binaural sound **localizes the "plop-plop-plop" in 3D space** relative to the user

Stereo Sound vs 3D Sound

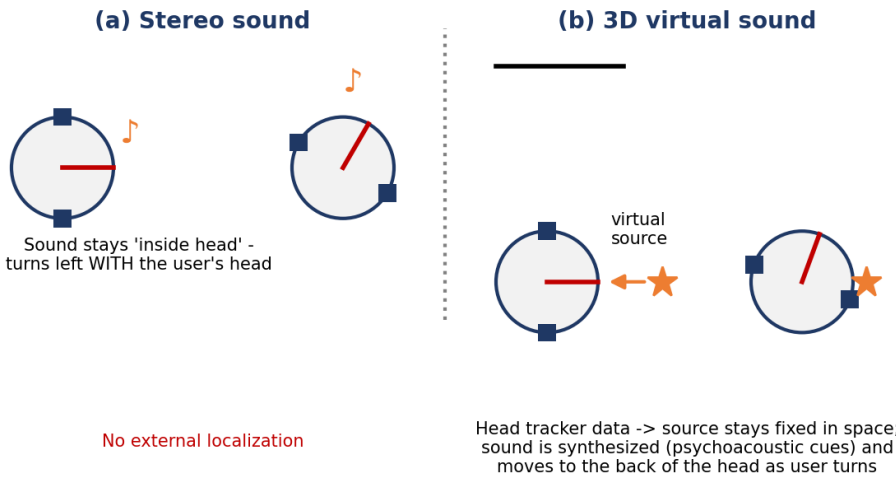


Fig C.1 Stereo sound versus 3D virtual sound

	Stereo sound	3D (virtual) sound
Localization	Seems to emanate **inside the user's head — not externalized**, as real sounds are	Perceived as coming from the **user's surroundings** (external)
When user turns left	The violin **turns left following the head motion**	The virtual violin **remains localized in space**; its sound appears to move to the back of the head
Information	Plain left/right signal	Contains significant **psychoacoustic information**; sound is **synthesized** using **head tracker data**
Room realism	—	Reflected sounds from walls, floor and ceiling must be factored in

> **KEY POINT:** Highly immersive VR simulations should have **3D sound (virtual sound)** in addition to graphics feedback.

The Human Auditory System — Vertical-Polar Coordinate System

- 3D sound displays cannot be effective without understanding how we **localize sound sources in space**.
- To measure position, a **head-attached system of coordinates** is needed. If Cartesian, the sound-source location is the triad **(x, y, z)**.
- Alternatively it can be expressed in a spherical system called the **vertical-polar coordinate system**: the location is uniquely determined by **3 variables — azimuth, elevation and range**.

Variable	Definition	Range
Azimuth (θ)	Angle between the **nose** and a **plane containing the source and the vertical z axis**	±180°

Variable	Definition	Range
Elevation (ϕ)	Angle made with the **horizontal plane** by a line passing through the source and the **center of head**	±90°
Range (r)	**Distance to the source** measured along this line	Positive and larger than head radius (simplified spherical head model)

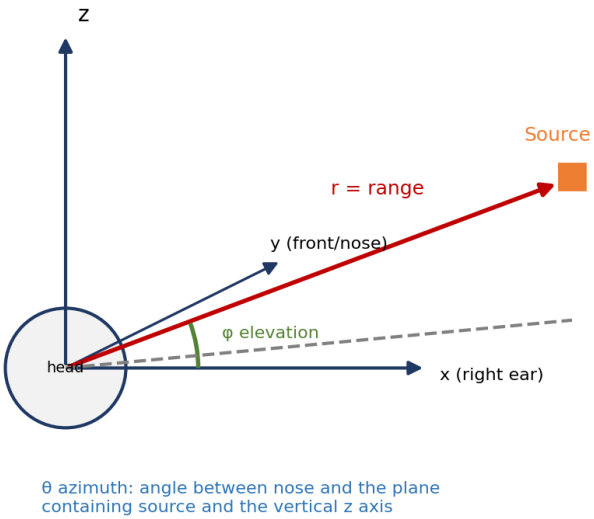


Fig C.2 Vertical-polar coordinate system used to locate 3D sound sources

- The **brain estimates the source location** (azimuth, elevation, range) based on **intensity, frequency, and temporal cues** present in the sound perceived by the left and right ears.

Azimuth Cue — Interaural Time Difference (ITD) [textbook, partly legible in scan]

The head has a fixed propagation delay: the ear closest to the source hears the sound first; the other ear's wave must travel an extra distance. The difference in arrival time at the two ears is the **interaural time difference**:

```
ITD = (a / c) * (θ + sin θ)

a = head radius,  c = speed of sound,  θ = source azimuth
ITD is maximum when θ = 90° and zero when the source is directly in front of or behind
the head
```

> **KEY POINT:** Mini summary: **Monaural** = same sound both ears; **binaural** = different sound per ear (needed when source can leave the FOV). **3D sound** = binaural + head-tracked + psychoacoustic synthesis → sound is *externalized* and fixed in space.

C.3 Haptic Feedback

- Named after the Greek term **hapthai** meaning **touch**.
- They convey important sensorial information that helps users achieve **tactile identification of virtual objects** in the environment and move these objects to perform a task.

- When added to visual and 3D audio feedback, haptic feedback **greatly improves simulation realism**.
- It becomes **mandatory** whenever visual feedback is **corrupted** (**occluded objects** that need to be manipulated) or **lacking entirely** (**dark environments**).
- The feedback is **limited to the hand and wrist** because the **hand has the highest density of touch receptors** in the body. Whole-body feedback systems are either at research stage or too specialized.
- **Touch and force**, two modalities are considered (the literature intermixes the terms, so clarification is needed). Example: a hand pushing lightly on a desk is first sensed by **fingertip touch sensors**; pushing harder contracts the muscles of hand and forearm and the force is felt by sensors on muscle ligaments and bones (**kinesthesis**).

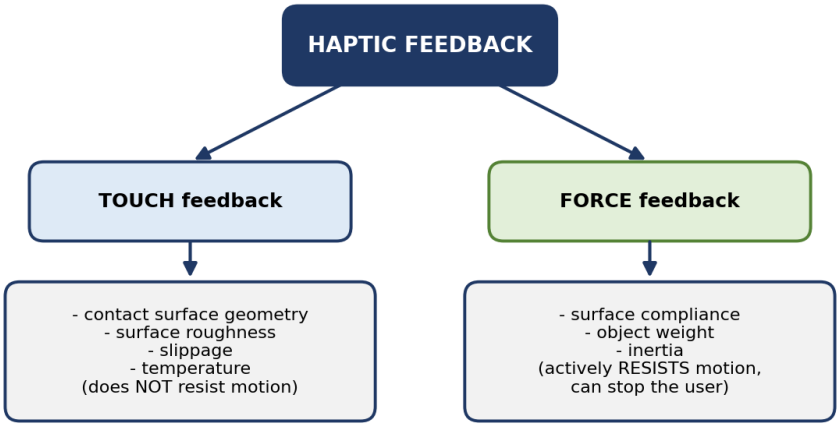


Fig C.3 Haptic feedback: touch vs force modalities

Touch Feedback vs Force Feedback

	Touch feedback	Force feedback
Conveys (real-time information on)	Contact surface **geometry**, virtual object surface **roughness, slippage and temperature**	Virtual object surface **compliance, object weight and inertia**
Effect on user's motion	Does **not actively resist** the user's contact motion; **cannot stop** the user from moving through virtual surfaces	**Actively resists** the user's contact motion and **can stop it** (for large feedback forces)
Sensed by	Fingertip touch (tactile) sensors	Muscle/ligament/bone sensors (kinesthesis)

Challenges of Haptic Feedback (Design Requirements)

Designing good haptic feedback interfaces is a daunting task in view of the specific requirements they need to meet:

Challenge	Explanation
1. User safety and comfort	While the user interacts with virtual objects, **the forces he or she feels are real**. Contact forces need to be large (rigid objects) but **not large enough to harm the user**. Good design is **fail-safe**, so that users are not subject to accidents if the system (computer) fails.

Challenge	Explanation
2. Portability and user comfort	Must provide sufficient force while keeping the feedback hardware **light and unintrusive**. If **too heavy and bulky**, the user gets tired easily and prefers less cumbersome open-loop control. Heavy structures can be gravity counterbalanced, but this **increases complexity and cost**. Portability also refers to **ease of use and installation**.
3. Complexity and cost	Counterbalancing and extra actuators increase both.
4. Self-contained	Should **not require special supporting construction, piping, or wiring**.

All these conflicting requirements call for a **compromise**, which can only be found if the **human haptic characteristics** are taken into account:

Specialist	Interested in
Hardware designer	Manual **force exertion capability** (selects feedback actuators); manual **degrees of freedom and mobility** (number of actuators, range of motion)
Control engineer	**Sensorial bandwidth** of the hand → how fast the control loop needs to be closed
Human factors specialist	**Safety and comfort** issues of feedback to the hand

The Human Haptic System

- **Input** to the human haptic system is provided by the **sensing loop**; **output** to the environment (the haptic interface) is mediated by the **sensory-motor control loop**.
- Input data are gathered by a multitude of **tactile, proprioceptive and thermal sensors**; output is **forces and torques resulting from muscular exertion**.
- The system is **not balanced**: humans perceive hapticly **much faster than they can respond**.

Haptic Sensing — Skin Mechanoreceptors

The skin houses **four types of tactile sensors**: **Meissner corpuscles (the majority), Merkel disks, Pacinian corpuscles, and Ruffini corpuscles**. When excited they produce small electrical discharges which are sensed by the brain.

- **Slow-adapting (SA)** (Merkel, Ruffini): maintain a constant rate of discharge for a long time in response to a constant force on the skin → used for **quasi-steady-state contact** (edges, skin stretch, static force).
- **Fast-adapting (FA)** (Meissner, Pacinian): drop their discharge rate so fast that a constant contact force becomes undetected → needed when contact has **strong time variation** (vibrations, accelerations).
- Merkel & Ruffini are most sensitive to **low-frequency stimuli (0–10 Hz)**; Meissner & Pacinian respond to **higher frequencies (50–300 Hz)**.
- The **receptive field** of a receptor determines its **spatial resolution**: large field → low resolution; Meissner & Merkel (small fields) → high spatial resolution.

Receptor type	Rate of adaptation	Stimulus frequency (Hz)	Receptive field	Detection function
Merkel disks	SA-I	0–10	Small, well defined	Edges, intensity
Ruffini corpuscles	SA-II	0–10	Large, indistinct	Static force, skin stretch
Meissner corpuscles	FA-I	20–50	Small, well defined	Velocity, edges
Pacinian corpuscles	FA-II	100–300	Large, indistinct	Acceleration, vibration

- **Spatial resolution of the skin** varies with receptor density: **fingertips** can discriminate two contacts at least **2.5 mm** apart; the **palm** cannot discriminate points less than **11 mm** apart.
- **Temporal resolution:** skin mechanoreceptors have a successiveness threshold of only **5 msec** (the eye: 25 msec).
- **Temperature sensing** is by **thermoreceptors** (epidermis — sensitive to cold; dermis — sensitive to warmth) and nociceptors; their spatial resolution is less than that of mechanoreceptors.

KEY POINT: Mini summary: **Haptic = touch + force**. Touch gives *what the surface is like* (geometry, roughness, slippage, temperature) and cannot stop you; force gives *how it resists* (compliance, weight, inertia) and can. Design must trade off **safety, comfort, portability, cost, self-containment**.

QUICK REVISION SHEET

Topic	One-line recall
Sketchpad paradigm	See object → pick it → object changes → repeat (Sutherland, 1963)
Physical vs logical device	Hardware vs what the API returns (position, ID, string…)
6 logical devices (GKS/PHIGS)	Locator, Pick, Keyboard, Stroke, Valuator, Choice
Trigger / measure	Trigger fires the device; measure is the data returned
Request vs event mode	Program waits for trigger vs events queued at arbitrary times
Event types	Window, Mouse, Motion, Keyboard, Idle
GLUT callbacks	Display, Mouse, Motion, PassiveMotion, Keyboard, Reshape, Idle (+ menu)
Mouse y-fix	`y = h - y` (pixels top-left → OpenGL bottom-left)
Menu steps	Define entries → define actions → attach to a mouse button
6 DOF	3 translations + 3 rotations (yaw, pitch, roll)
Tracker parameters	Accuracy, jitter, drift, latency
Absolute vs relative	Absolute never zero at rest; relative returns zeros if not acted on
Cubic Mouse	3 rods + 6 buttons, dual-hand, cutting planes, 9 cm cube, Fastrak
Trackball	3 forces + 3 torques, spring law, 6 LEDs + 6 photo sensors, RS232, gains
Gesture interface	Sensing gloves; larger work envelope; multipoint, dextrous
Sound display	Monaural vs binaural; 3D sound externalized & head-tracked
Vertical-polar system	Azimuth (±180°), elevation (±90°), range
Haptic	Touch (geometry, roughness, slippage, temperature) vs Force (compliance, weight, inertia)
Haptic challenges	Safety, portability/comfort, complexity & cost, self-contained

Likely Exam Questions

1. Explain physical and logical input devices with examples. List the six logical device types.
2. Differentiate between request mode and event mode. What are measure and trigger?
3. Explain the GLUT event loop and callbacks. Write a program to draw a square at the mouse position.
4. Explain the keyboard, mouse (pointing device) and reshape callbacks; why must the y coordinate be inverted?
5. How are menus created in GLUT? Write the code with a submenu.
6. What are 3D trackers? Explain the tracker performance parameters (accuracy, jitter, drift, latency) with diagrams.
7. Explain tracker-based navigation interfaces: 3Ball, wand, Cubic Mouse. Compare trackers and trackballs.

8. What are gesture interfaces? Explain sensing gloves and their parameters.
9. Differentiate stereo sound and 3D sound. Explain the vertical-polar coordinate system.
10. Explain haptic feedback: touch vs force feedback, and the challenges in designing haptic interfaces.